Exercício 10.2: Scan de vulnerabilidades e correção da imagem base

Aluno: Luiz Brito · **Data:** 24/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Docker rootless, Trivy **v0.75.0**. Instalado do release oficial no GitHub com o SHA-256 conferido contra o `checksums.txt`. Foi escolhida uma versão com algumas semanas de uso por causa do comprometimento da cadeia de suprimentos do Trivy de 21/03/2026 (GHSA publicado pela Aqua).
- Base de vulnerabilidades atualizada pelo Trivy em 24/10/2026. Os números mudam a cada dia, porque a base muda.
- Sequência completa em `01-commands.sh`.

Antes: `store-api:vuln`

```
# versao deliberadamente ruim: imagem completa, sem multi-stage, root, COPY . . sem .dockerignore
FROM python:3.12
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

O Docker aplica `<Dockerfile>.dockerignore` no lugar do `.dockerignore` da pasta, e `Dockerfile.vuln.dockerignore` está vazio de propósito. Assim o `COPY . .` reproduz o "sem `.dockerignore`" mesmo com o `.dockerignore` da versão corrigida no mesmo diretório. Dependências usadas no build: as originais do app (`fastapi==0.115.6`, `uvicorn[standard]==0.34.0`).

```
$ docker run --rm store-api:vuln id
uid=0(root) gid=0(root) groups=0(root)

$ docker run --rm store-api:vuln ls -la /app
-rw-rw-r-- 1 root root  240 Oct 24 04:03 Dockerfile.vuln
-rw-rw-r-- 1 root root 2006 Oct 24 03:35 README.md
-rw-rw-r-- 1 root root  963 Oct 24 04:03 main.py
-rw-rw-r-- 1 root root   43 Oct 24 04:03 requirements.txt
```

O `COPY . .` levou para a imagem o `Dockerfile` e o `README`, que não têm nada a ver com a execução. Aqui não havia segredo na pasta. Se houvesse um `.env` ou um `.git`, eles teriam ido junto do mesmo jeito.

```
$ trivy image --severity HIGH,CRITICAL store-api:vuln > relatorio-antes.txt

store-api:vuln (debian 13.7)
Total: 460 (HIGH: 436, CRITICAL: 24)

Python (python-pkg)
Total: 3 (HIGH: 3, CRITICAL: 0)
| starlette | CVE-2025-62727 | HIGH | fixed | 0.41.3 | 0.49.1 | Starlette DoS via Range header
merging
|           | CVE-2026-48818 |      |       |        | 1.1.0  | SSRF and NTLM credential theft via
UNC paths in StaticFiles
|           | CVE-2026-54283 |      |       |        | 1.3.1  | request.form() limits silently
ignored ... DoS
```

Depois: `store-api:1.1`

Dependências atualizadas para as versões corrigidas. A `starlette` chega via `fastapi`. A correção exige `⩾ 1.3.1`, e ela foi fixada explicitamente porque o `fastapi` aceita qualquer `⩾ 0.46.0`:

```
fastapi==0.162.0
starlette==1.9.0
uvicorn[standard]==0.54.0
```

```
# store-api endurecida: multi-stage + distroless nonroot
# ---------- build ----------
# mesmo Debian 12 / Python 3.11 da imagem final: o venv aponta para /usr/bin/python3,
# que existe nas duas (um builder python:3.12-slim gera um venv que nao roda no distroless)
FROM debian:12-slim AS builder
RUN apt-get update \
 && apt-get install -y --no-install-recommends python3-venv \
 && rm -rf /var/lib/apt/lists/*
RUN python3 -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
# pip e setuptools so servem para instalar; o setuptools 66 do Debian tem CVE HIGH
RUN pip install --no-cache-dir -r requirements.txt \
 && pip uninstall -y pip setuptools

# ---------- runtime ----------
FROM gcr.io/distroless/python3-debian12:nonroot
ARG APP_VERSION=1.1
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
COPY --chown=65532:65532 main.py .
ENV PATH="/opt/venv/bin:$PATH" \
    PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    APP_VERSION=$APP_VERSION
USER 65532:65532
EXPOSE 8000
ENTRYPOINT ["/opt/venv/bin/python3", "-m", "uvicorn", "main:app", "--host", "0.0.0.0", "--port",
"8000"]
```

```
.git
.env
*.pem
*.key
tests/
__pycache__/
Dockerfile*
relatorio-*.txt
*.json
*.log
*.md
```

Achado: o Dockerfile da seção 10.4.2 da apostila não roda. Testado sem alterações (`Dockerfile.apostila`):

```
$ docker run --rm store-api:apostila
/usr/bin/python: No module named uvicorn
```

O builder `python:3.12-slim` cria um venv cujo `python` é um symlink para `/usr/local/bin/python3.12`. Esse caminho não existe no distroless `python3-debian12`, que traz o Python **3.11** em `/usr/bin/python3`:

```
$ docker run --rm --entrypoint python3 gcr.io/distroless/python3-debian12:nonroot -c "import sys;
print(sys.version)"
3.11.2 (main, Apr 28 2025, 14:11:48) [GCC 12.2.0]
```

Com o symlink quebrado, o `python` do `PATH` cai no `/usr/bin/python` do sistema, que não enxerga o venv. A correção é construir no **mesmo** Debian 12 e com o mesmo Python da imagem final (`debian:12-slim` + `python3-venv`). Também foram removidos `pip` e `setuptools` do venv no fim do build: o `setuptools 66.1.1` que vem do Debian tinha 2 CVEs HIGH (CVE-2024-6345 e CVE-2025-47273) e não tem uso em tempo de execução.

```
$ docker run -d --name t11 store-api:1.1
$ docker exec t11 id
OCI runtime exec failed: exec failed: unable to start container process: exec: "id": executable file
not found in $PATH
$ docker exec t11 python3 -c "import os; print('uid', os.getuid())"
uid 65532
$ docker top t11 -eo pid,user,uid,args
PID      USER    UID     COMMAND
780529   65532   65532   /opt/venv/bin/python3 -m uvicorn main:app --host 0.0.0.0 --port 8000
```

Sem `id` nem shell, que é o objetivo do distroless. O UID foi conferido com o próprio Python e com o `docker top` visto do host.

```
$ trivy image --severity HIGH,CRITICAL store-api:1.1 > relatorio-depois.txt

store-api:1.1 (debian 12.13)
Total: 63 (HIGH: 61, CRITICAL: 2)
| libsqlite3-0 | CVE-2025-7458  | CRITICAL | affected     | 3.40.1-2+deb12u2 |  (sem correção)
| zlib1g       | CVE-2023-45853 | CRITICAL | will_not_fix | 1:1.2.13.dfsg-1  |  (Debian não vai
corrigir)

(nenhuma vulnerabilidade em pacotes Python)
```

```
$ trivy image --format cyclonedx -o store-api-sbom.json store-api:1.1
$ jq '.components | length' store-api-sbom.json
55
$ jq -r '.bomFormat + " " + .specVersion' store-api-sbom.json
CycloneDX 1.7
```

Tabela comparativa

	`store-api:vuln`	`store-api:1.1`	Variação
Tamanho (`docker images`, em disco)	1,7 GB	134 MB	**-92%**
Tamanho (`image inspect .Size`, conteúdo)	440,0 MB	31,7 MB	**-93%**
CRITICAL (total)	24	2	-92%
HIGH (total)	439 (436 SO + 3 Python)	61	-86%
CRITICAL **corrigíveis** (`--ignore-unfixed`)	0	**0**	igual
HIGH **corrigíveis** (`--ignore-unfixed`)	5 (2 SO + 3 Python)	**44**	**+39**
Pacotes no SBOM (CycloneDX)	489	55	-89%
UID do processo	0 (root)	65532	

Achado crítico: menos CVEs no total, mais CVEs corrigíveis

Os totais caem muito, mas os **HIGH corrigíveis subiram de 5 para 44**. A `store-api:vuln` usa `python:3.12`, que hoje é Debian **13.7** e é reconstruída com frequência, então quase tudo o que tem correção já está corrigido nela. O distroless `python3-debian12` é Debian **12.13**, e os 44 HIGH estão em pacotes que o Debian 12 já corrigiu mas que ainda não entraram na imagem distroless (`libexpat1` 2.5.0-1+deb12u2 → deb12u5, `libkrb5-3` deb12u4 → deb12u5, `libpython3.11` deb12u6 → deb12u7). No distroless não dá para aplicar a correção com `apt-get upgrade`, porque a imagem não tem gerenciador de pacotes. Só se resolve com uma nova base.

Comparação com a variante Debian 13 da mesma família, que não é a pedida no enunciado:

```
$ trivy image --severity HIGH,CRITICAL gcr.io/distroless/python3-debian13:nonroot
gcr.io/distroless/python3-debian13:nonroot (debian 13.7)
Total: 26 (HIGH: 26, CRITICAL: 0)
$ trivy image --severity HIGH,CRITICAL --ignore-unfixed --format json ... | jq
'[.Results[].Vulnerabilities[]?] | length'
0
```

A recomendação para produção é `python3-debian13`: 0 corrigíveis, e o builder passa a ser `debian:13-slim` com Python 3.13. A lição vai além da troca de imagem: "imagem mínima" reduz a superfície, mas a imagem também precisa ser **reconstruída com frequência**. Uma base pequena e desatualizada pode ter mais CVE corrigível do que uma base grande e atual.

Porta de qualidade no CI

```
trivy image --exit-code 1 --ignore-unfixed --severity CRITICAL,HIGH store-api:1.1
```

`--ignore-unfixed` faz o pipeline falhar só pelo que o time consegue resolver. Os 2 CRITICAL sem correção (`affected`, `will_not_fix`) não travam o deploy, mas continuam visíveis no relatório. Hoje esse comando **reprova** a `store-api:1.1` (`exit=1`, pelos 44 HIGH corrigíveis da base Debian 12). É o comportamento correto: a porta força a troca para a base atualizada. Só com `--severity CRITICAL` a imagem passaria (`exit=0`), e é por isso que HIGH faz parte da porta.

Critérios de aceite

Critério	Resultado
Redução de pelo menos 80% no tamanho	ok (-92% em disco, -93% em conteúdo)
Zero CRITICAL **corrigíveis** depois	ok (2 CRITICAL, ambos sem correção)
Processo com UID diferente de 0	ok (65532)
SBOM válido, `jq '.components | length'` > 0	ok (55)